



CRYPTOGRAPHY

Archana M

Department of Computer Applications

CRYPTOGRAPHY

Asymmetric-Key Cryptography and Key Management-Unit 3

Archana M

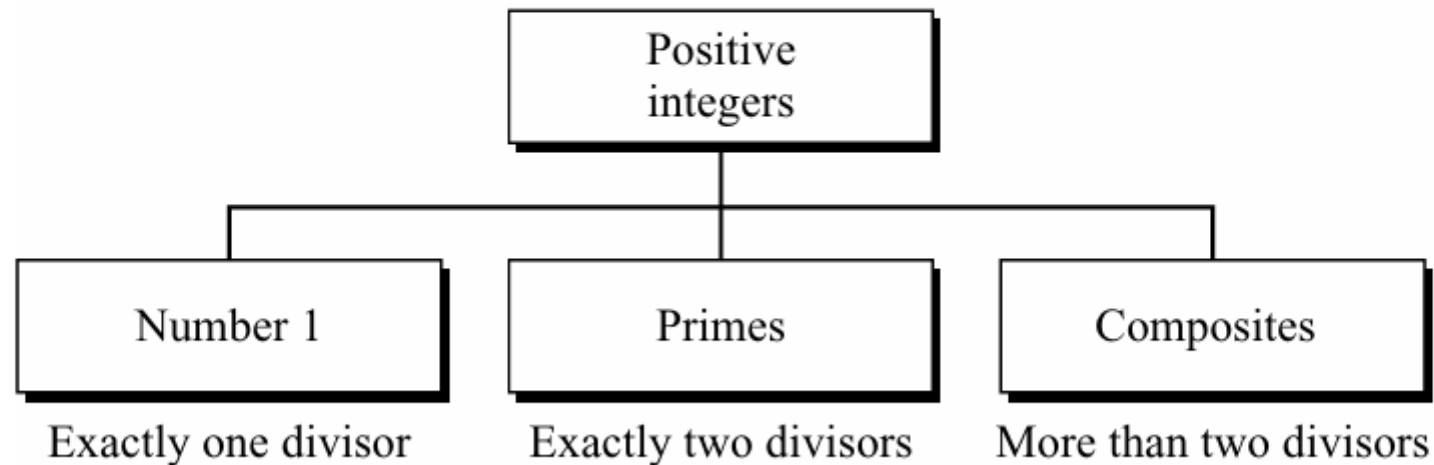
Department of Computer Applications

Mathematics of Cryptography

PRIMES

- Asymmetric-key cryptography uses primes extensively.
- The positive integers can be divided into three groups: the number 1, primes, and composites

Figure 9.1 *Three groups of positive integers*



-
- A positive integer is a prime if and only if it is exactly divisible by two integers, 1 and itself.
 - A composite is a positive integer with more than two divisors.

Example : What is the smallest prime?

The smallest prime is 2, which is divisible by 2 (itself) and 1.

Note that the integer 1 is not a prime according to the definition, because a prime must be divisible by two different integers, no more, no less. The integer 1 is divisible only by itself; it is not a prime

-
- Example 2: List the primes smaller than 10.

Solution : There are four primes less than 10: 2, 3, 5, and 7

Coprimes: Two positive integers, a and b , are relatively prime, or coprime, if $\gcd(a, b) = 1$

Note that the number 1 is relatively prime with any integer. If p is a prime, then all integers 1 to $p - 1$ are relatively prime to p .

Prime numbers are **very important in cryptography** because they help create **secure encryption algorithms**. Many modern cryptographic systems rely on mathematical properties of prime numbers that are **easy to compute in one direction but very difficult to reverse** without special information

- The main reasons why primes are useful.

Difficulty of Factorization (Security)

- One of the biggest reasons is the **prime factorization problem**.
- If we multiply two large prime numbers, it is **easy** to compute the product.
- But if someone only knows the product, it is **extremely difficult** to find the original primes.

Example:

p=61 q=53

Multiply:

$$\mathbf{n=p \times q = 61 * 53 = 3233}$$

Finding **3233 from 61 and 53 is easy**, but finding **61 and 53 from 3233 is hard** (when numbers are very large).

Relatively prime/Coprime

- Two numbers are **relatively prime** (or **coprime**) if their **greatest common divisor (GCD)** is 1.
- This means the two numbers **do not share any common factor except 1**.

Mathematical Definition

*Two integers **a** and **b** are relatively prime if $\gcd(a,b)=1$*

Example 1: Check if **8 and 15** are relatively prime.

- Factors of **8**
1, 2, 4, 8
- Factors of **15**
1, 3, 5, 15
- Common factor = **1**
- $\gcd(8,15)=1$

8 and 15 are relatively prime

Euler's Phi-Function

- Euler's phi-function, $\phi(n)$, which is sometimes called the Euler's totient function plays a very important role in cryptography.
- The function finds the number of integers that are both smaller than n and relatively prime to n .
- The function $\phi(n)$ calculates the number of elements in this set

Example1: What is the value of $\phi(13)$?

Solution: Because 13 is a prime, $\phi(13) = (13 - 1) = 12$.

Example2: What is the value of $\phi(8)$?

Numbers ≤ 8 : 1, 2, 3, 4, 5, 6, 7, 8

- Check which are **coprime with 8** - Coprime numbers: **1, 3, 5, 7**

$$\varphi(8)=4$$

Important Properties

1. If p is a prime number

$$\varphi(p) = p - 1$$

- Example:

$\varphi(11) = 10$ because all numbers **1 to 10** are relatively prime to 11.

2. If p and q are prime numbers

$$\varphi(pq) = (p - 1)(q - 1)$$

- Example:

$$p = 3, q = 5$$

$$\varphi(15) = (3 - 1)(5 - 1) = 2 \times 4 = 8$$

Important in Cryptography

Euler's phi function is very important in the **RSA algorithm**.

- Steps where it is used:

1. Choose two prime numbers p and q

2. Compute $n=pq$

3. Compute

$\varphi(n)=(p-1)(q-1)$ to generate **public key and private key**.

CRYPTOGRAPHY

Asymmetric-Key Cryptography and Key Management-Unit 3

Archana M

Department of Computer Applications

Chinese Remainder Theorem

The Chinese remainder theorem (CRT) is used to solve a set of congruent equations with one variable but different moduli, which are relatively prime, as shown below:

$$\begin{aligned}x &\equiv a_1 \pmod{m_1} \\x &\equiv a_2 \pmod{m_2} \\&\dots \\x &\equiv a_k \pmod{m_k}\end{aligned}$$

The Chinese remainder theorem states that the above equations have a unique solution if the moduli are relatively prime.

Chinese Remainder Theorem

Example: The following is an example of a set of equations with different moduli:

$$x \equiv 2 \pmod{3}$$

$$x \equiv 3 \pmod{5}$$

$$x \equiv 2 \pmod{7}$$

Chinese Remainder Theorem

Solution

The solution to the set of equations follows these steps:

1. Find $M = m_1 \times m_2 \times \dots \times m_k$. This is the common modulus.
2. Find $M_1 = M/m_1, M_2 = M/m_2, \dots, M_k = M/m_k$.
3. Find the multiplicative inverse of $M_1, M_2, \dots, M_k$ using the corresponding moduli ($m_1, m_2, \dots, m_k$). Call the inverses $M_1^{-1}, M_2^{-1}, \dots, M_k^{-1}$.
4. The solution to the simultaneous equations is

$$x = (a_1 \times M_1 \times M_1^{-1} + a_2 \times M_2 \times M_2^{-1} + \dots + a_k \times M_k \times M_k^{-1}) \bmod M$$

Chinese Remainder Theorem

The Chinese Remainder Theorem

Given		To Find	
$a_1 = 2$	$m_1 = 3$	M_1	M_1^{-1}
$a_2 = 3$	$m_2 = 5$	M_2	M_2^{-1}
$a_3 = 2$	$m_3 = 7$	M_3	M_3^{-1}

$M=105$

Solution:

$$M = m_1 \times m_2 \times m_3$$

$$M = 3 \times 5 \times 7$$

$$M = 105$$

nesoacademy.org

The Chinese Remainder Theorem

Given		To Find	
$a_1 = 2$	$m_1 = 3$	$M_1 = 35$	M_1^{-1}
$a_2 = 3$	$m_2 = 5$	$M_2 = 21$	M_2^{-1}
$a_3 = 2$	$m_3 = 7$	$M_3 = 15$	M_3^{-1}

$M=105$

$$M_1 = \frac{M}{m_1}$$

$$M_1 = \frac{105}{3}$$

$$M_1 = 35$$

$$M_2 = \frac{M}{m_2}$$

$$M_2 = \frac{105}{5}$$

$$M_2 = 21$$

$$M_3 = \frac{M}{m_3}$$

$$M_3 = \frac{105}{7}$$

$$M_3 = 15$$

nesoacademy.org

Chinese Remainder Theorem

The Chinese Remainder Theorem

Given		To Find	
$a_1 = 2$	$m_1 = 3$	$M_1 = 35$	$M_1^{-1} = 2$
$a_2 = 3$	$m_2 = 5$	$M_2 = 21$	$M_2^{-1} = 1$
$a_3 = 2$	$m_3 = 7$	$M_3 = 15$	$M_3^{-1} = 1$

$M=105$

$$\begin{aligned}
 M_1 \times M_1^{-1} &= 1 \pmod{m_1} \\
 35 \times M_1^{-1} &= 1 \pmod{3} \\
 35 \times 2 &= 1 \pmod{3} \\
 M_1^{-1} &= 2
 \end{aligned}$$

$$\begin{aligned}
 M_2 \times M_2^{-1} &= 1 \pmod{m_2} \\
 21 \times M_2^{-1} &= 1 \pmod{5} \\
 21 \times 1 &= 1 \pmod{5} \\
 M_2^{-1} &= 1
 \end{aligned}$$

$$\begin{aligned}
 M_3 \times M_3^{-1} &= 1 \pmod{m_3} \\
 15 \times M_3^{-1} &= 1 \pmod{7} \\
 15 \times 1 &= 1 \pmod{7} \\
 M_3^{-1} &= 1
 \end{aligned}$$

nesoacademy.org

The Chinese Remainder Theorem

Example 1: Solve the following equations using CRT

$$X \equiv 2 \pmod{3}$$

$$X \equiv 3 \pmod{5}$$

$$X \equiv 2 \pmod{7}$$

Solution:

$a_1 = 2$	$m_1 = 3$	$M_1 = 35$	$M_1^{-1} = 2$	$M=105$
$a_2 = 3$	$m_2 = 5$	$M_2 = 21$	$M_2^{-1} = 1$	
$a_3 = 2$	$m_3 = 7$	$M_3 = 15$	$M_3^{-1} = 1$	

$$\begin{aligned}
 X &= (a_1 M_1 M_1^{-1} + a_2 M_2 M_2^{-1} + a_3 M_3 M_3^{-1}) \pmod{M} \\
 &= (2 \times 35 \times 2 + 3 \times 21 \times 1 + 2 \times 15 \times 1) \pmod{105} \\
 &= 233 \pmod{105}
 \end{aligned}$$

$$X = 23$$

Chinese Remainder Theorem

Note that the answer to this set of equations is $x = 23$. This value satisfies all equations: $23 \equiv 2 \pmod{3}$, $23 \equiv 3 \pmod{5}$, and $23 \equiv 2 \pmod{7}$.

Solution

From the previous example, we already know that the answer is $x = 23$. We follow the four steps.

1. $M = 3 \times 5 \times 7 = 105$
2. $M_1 = 105 / 3 = 35$, $M_2 = 105 / 5 = 21$, $M_3 = 105 / 7 = 15$
3. The inverses are $M_1^{-1} = 2$, $M_2^{-1} = 1$, $M_3^{-1} = 1$
4. $x = (2 \times 35 \times 2 + 3 \times 21 \times 1 + 2 \times 15 \times 1) \bmod 105 = 23 \bmod 105$

CRYPTOGRAPHY

Asymmetric-Key Cryptography and Key Management-Unit 3

Archana M

Department of Computer Applications

Chinese Remainder Theorem

The Chinese remainder theorem (CRT) is used to solve a set of congruent equations with one variable but different moduli, which are relatively prime, as shown below:

$$\begin{aligned}x &\equiv a_1 \pmod{m_1} \\x &\equiv a_2 \pmod{m_2} \\&\dots \\x &\equiv a_k \pmod{m_k}\end{aligned}$$

The Chinese remainder theorem states that the above equations have a unique solution if the moduli are relatively prime.

Chinese Remainder Theorem

Example: The following is an example of a set of equations with different moduli:

$$x \equiv 2 \pmod{3}$$

$$x \equiv 3 \pmod{5}$$

$$x \equiv 2 \pmod{7}$$

Chinese Remainder Theorem

Solution

The solution to the set of equations follows these steps:

1. Find $M = m_1 \times m_2 \times \dots \times m_k$. This is the common modulus.
2. Find $M_1 = M/m_1, M_2 = M/m_2, \dots, M_k = M/m_k$.
3. Find the multiplicative inverse of $M_1, M_2, \dots, M_k$ using the corresponding moduli ($m_1, m_2, \dots, m_k$). Call the inverses $M_1^{-1}, M_2^{-1}, \dots, M_k^{-1}$.
4. The solution to the simultaneous equations is

$$x = (a_1 \times M_1 \times M_1^{-1} + a_2 \times M_2 \times M_2^{-1} + \dots + a_k \times M_k \times M_k^{-1}) \bmod M$$

Chinese Remainder Theorem

The Chinese Remainder Theorem

Given		To Find	
$a_1 = 2$	$m_1 = 3$	M_1	M_1^{-1}
$a_2 = 3$	$m_2 = 5$	M_2	M_2^{-1}
$a_3 = 2$	$m_3 = 7$	M_3	M_3^{-1}

$M=105$

Solution:

$$M = m_1 \times m_2 \times m_3$$

$$M = 3 \times 5 \times 7$$

$$M = 105$$

nesoacademy.org

The Chinese Remainder Theorem

Given		To Find	
$a_1 = 2$	$m_1 = 3$	$M_1 = 35$	M_1^{-1}
$a_2 = 3$	$m_2 = 5$	$M_2 = 21$	M_2^{-1}
$a_3 = 2$	$m_3 = 7$	$M_3 = 15$	M_3^{-1}

$M=105$

$$M_1 = \frac{M}{m_1}$$

$$M_1 = \frac{105}{3}$$

$$M_1 = 35$$

$$M_2 = \frac{M}{m_2}$$

$$M_2 = \frac{105}{5}$$

$$M_2 = 21$$

$$M_3 = \frac{M}{m_3}$$

$$M_3 = \frac{105}{7}$$

$$M_3 = 15$$

nesoacademy.org

Chinese Remainder Theorem

The Chinese Remainder Theorem

Given		To Find	
$a_1 = 2$	$m_1 = 3$	$M_1 = 35$	$M_1^{-1} = 2$
$a_2 = 3$	$m_2 = 5$	$M_2 = 21$	$M_2^{-1} = 1$
$a_3 = 2$	$m_3 = 7$	$M_3 = 15$	$M_3^{-1} = 1$

$M=105$

$$M_1 \times M_1^{-1} = 1 \pmod{m_1}$$

$$35 \times M_1^{-1} = 1 \pmod{3}$$

$$35 \times 2 = 1 \pmod{3}$$

$$M_1^{-1} = 2$$

$$M_2 \times M_2^{-1} = 1 \pmod{m_2}$$

$$21 \times M_2^{-1} = 1 \pmod{5}$$

$$21 \times 1 = 1 \pmod{5}$$

$$M_2^{-1} = 1$$

$$M_3 \times M_3^{-1} = 1 \pmod{m_3}$$

$$15 \times M_3^{-1} = 1 \pmod{7}$$

$$15 \times 1 = 1 \pmod{7}$$

$$M_3^{-1} = 1$$

nesoacademy.org

The Chinese Remainder Theorem

Example 1: Solve the following equations using CRT

$$X \equiv 2 \pmod{3}$$

$$X \equiv 3 \pmod{5}$$

$$X \equiv 2 \pmod{7}$$

Solution:

$a_1 = 2$	$m_1 = 3$	$M_1 = 35$	$M_1^{-1} = 2$	$M=105$
$a_2 = 3$	$m_2 = 5$	$M_2 = 21$	$M_2^{-1} = 1$	
$a_3 = 2$	$m_3 = 7$	$M_3 = 15$	$M_3^{-1} = 1$	

$$X = (a_1 M_1 M_1^{-1} + a_2 M_2 M_2^{-1} + a_3 M_3 M_3^{-1}) \pmod{M}$$

$$= (2 \times 35 \times 2 + 3 \times 21 \times 1 + 2 \times 15 \times 1) \pmod{105}$$

$$= 233 \pmod{105}$$

$$X = 23$$

Chinese Remainder Theorem

Note that the answer to this set of equations is $x = 23$. This value satisfies all equations: $23 \equiv 2 \pmod{3}$, $23 \equiv 3 \pmod{5}$, and $23 \equiv 2 \pmod{7}$.

Solution

From the previous example, we already know that the answer is $x = 23$. We follow the four steps.

1. $M = 3 \times 5 \times 7 = 105$
2. $M_1 = 105 / 3 = 35$, $M_2 = 105 / 5 = 21$, $M_3 = 105 / 7 = 15$
3. The inverses are $M_1^{-1} = 2$, $M_2^{-1} = 1$, $M_3^{-1} = 1$
4. $x = (2 \times 35 \times 2 + 3 \times 21 \times 1 + 2 \times 15 \times 1) \pmod{105} = 23 \pmod{105}$

CRYPTOGRAPHY

Asymmetric-Key Cryptography and Key Management-Unit 3

Archana M

Department of Computer Applications

-
- Till now we emphasized the principles of symmetric-key cryptography. Now we start the discussion of asymmetric-key cryptography.
 - Symmetric and asymmetric-key cryptography will exist in parallel and continue to serve the community.
 - We actually believe that they are complements of each other;
 - the advantages of one can compensate for the disadvantages of the other.

-
- The conceptual differences between the two systems are based on how these systems keep a secret.
 - In symmetric-key cryptography, the secret must be shared between two persons.
 - In asymmetric-key cryptography, the secret is personal (unshared); each person creates and keeps his or her own secret.
 - In a community of n people, $n(n - 1)/2$ shared secrets are needed for symmetric-key cryptography;
 - Only n personal secrets are needed in asymmetric-key cryptography.

-
- For a community with a population of 1 million, symmetric-key cryptography would require half a billion shared secrets;
 - asymmetric-key cryptography would require 1 million personal secrets.
 - *Symmetric-key cryptography is based on sharing secrecy;*
 - *Asymmetric-key cryptography is based on personal secrecy.*

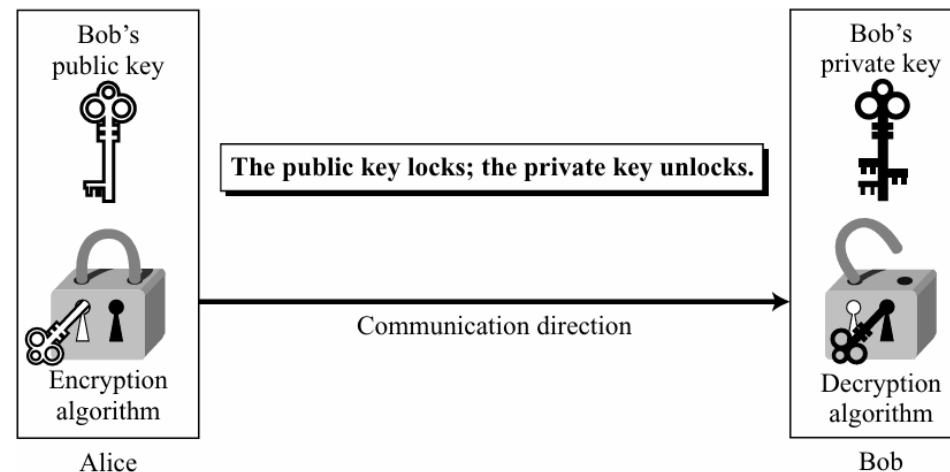
-
- There are some other aspects of security besides encipherment that need asymmetric key cryptography.
 - These include authentication and digital signatures.
 - Whenever an application is based on a personal secret, we need to use asymmetric-key cryptography.
 - Whereas symmetric-key cryptography is based on substitution and permutation of symbols (characters or bits), asymmetric-key cryptography is based on applying mathematical functions to numbers

-
- In symmetric-key cryptography, the plaintext and cipher text are thought of as a combination of symbols.
 - Encryption and decryption permute these symbols or substitute a symbol for another.
 - In asymmetric-key cryptography, the plaintext and ciphertext are numbers;
 - encryption and decryption are mathematical functions that are applied to numbers to create other numbers.

Keys

- Asymmetric key cryptography uses two separate keys: one private and one public.
- If encryption and decryption are thought of as locking and unlocking padlocks with keys, then the padlock that is locked with a public key can be unlocked only with the corresponding private key.

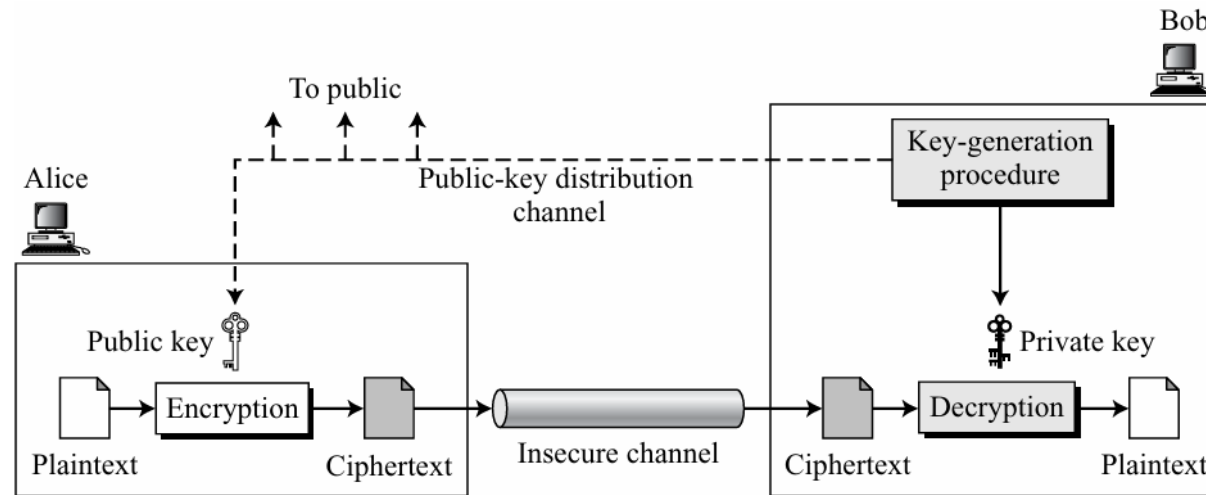
Figure 10.1 *Locking and unlocking in asymmetric-key cryptosystem*



General Idea

- Figure shows the general idea of asymmetric-key cryptography as used for encipherment
- The figure shows that, unlike symmetric-key cryptography, there are distinctive keys in asymmetric-key cryptography: a private key and a public key

Figure 10.2 *General idea of asymmetric-key cryptosystem*



-
- The Figure shows several important facts.
 - First, it emphasizes the asymmetric nature of the cryptosystem.
 - The burden of providing security is mostly on the shoulders of the receiver (Bob, in this case).
 - Bob needs to create two keys: one private and one public.
 - Bob is responsible for distributing the public key to the community.
 - This can be done through a public-key distribution channel.
 - Although this channel is not required to provide secrecy, it must provide authentication and integrity. Eve should not be able to advertise her public key to the community pretending that it is Bob's public key

-
- Second, asymmetric-key cryptography means that Bob and Alice cannot use the same set of keys for two-way communication.
 - Each entity in the community should create its own private and public keys.
 - The Figure shows how Alice can use Bob's public key to send encrypted messages to Bob.
 - If Bob wants to respond, Alice needs to establish her own private and public keys.

-
- Third, asymmetric-key cryptography means that Bob needs only one private key to receive all correspondence from anyone in the community,
 - But Alice needs n public keys to communicate with n entities in the community, one public key for each entity.
 - In other words, Alice needs a ring of public keys

Plaintext/Ciphertext

- Unlike in symmetric-key cryptography, plaintext and ciphertext are treated as integers in asymmetric-key cryptography
- The message must be encoded as an integer (or a set of integers) before encryption; the integer (or the set of integers) must be decoded into the message after decryption.
- Asymmetric-key cryptography is normally used to encrypt or decrypt small pieces of information, such as the cipher key for a symmetric key cryptography.
- In other words, asymmetric-key cryptography normally is used for ancillary goals instead of message encipherment.

Encryption/Decryption

- Encryption and decryption in asymmetric-key cryptography are mathematical functions applied over the numbers representing the plaintext and ciphertext
- The ciphertext can be thought of as $C = f(K_{\text{public}}, P)$;
- the plaintext can be thought of as $P = g(K_{\text{private}}, C)$.
- The encryption function f is used only for encryption; the decryption function g is used only for decryption.
- Next we show that the function f needs to be a trapdoor one-way function to allow Bob to decrypt the message but to prevent Eve from doing so.

Need for Both

- There is a very important fact that is sometimes misunderstood
- The advent of asymmetric key (public-key) cryptography does not eliminate the need for symmetric-key (secret key) cryptography.
- The reason is that asymmetric-key cryptography, which uses mathematical functions for encryption and decryption, is much slower than symmetric-key cryptography.
- For encipherment of large messages, symmetric-key cryptography is still needed.
- On the other hand, the speed of symmetric-key cryptography does not eliminate the need for asymmetric-key cryptography.

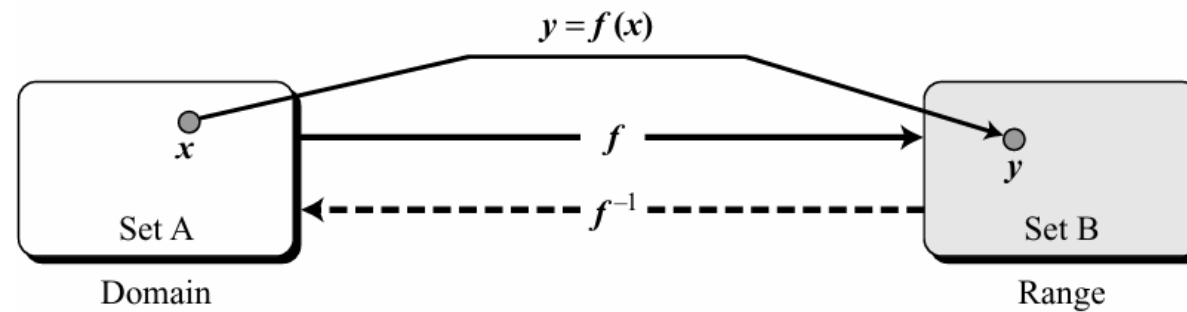
-
- Asymmetric-key cryptography is still needed for authentication, digital signatures, and secret-key exchanges.
 - This means that, to be able to use all aspects of security today, we need both symmetric-key and asymmetric-key cryptography.
 - One complements the other.

Trapdoor One-Way Function

- The main idea behind asymmetric-key cryptography is the concept of the trapdoor one way function.

Functions : A function is a rule that associates (maps) one element in set A, called the domain, to one element in set B, called the range, as shown in figure

Figure 10.3 *A function as rule mapping a domain to a range*



An invertible function is a function that associates each element in the range with exactly one element in the domain.

One way Function

One-Way Function

A **one-way function (OWF)** is a function that satisfies the following two properties:

1. f is easy to compute. In other words, given x , $y = f(x)$ can be easily computed.
2. f^{-1} is difficult to compute. In other words, given y , it is computationally infeasible to calculate $x = f^{-1}(y)$.

Trapdoor One-Way Function:

A **trapdoor one-way function (TOWF)** is a one-way function with a third property:

3. **Given y and a trapdoor (secret), x can be computed easily.**

- Example:

When n is large, $n = p \times q$ is a one-way function.

Note that in this function x is a tuple (p, q) of two primes and y is n .

Given p and q , it is always easy to calculate n ; given n , it is very difficult to compute p and q .

This is the factorization problem

Knapsack Cryptosystem

- The first brilliant idea of public-key cryptography came from Merkle and Hellman, in their knapsack cryptosystem.
- Although this system was found to be insecure with today's standards, the main idea behind this cryptosystem gives an insight into recent public-key cryptosystems
- If we are told which elements, from a predefined set of numbers, are in a knapsack, we can easily calculate the sum of the numbers;
- if we are told the sum, it is difficult to say which elements are in the knapsack.

- Definition

Suppose we are given two k -tuples, $a = [a_1, a_2, \dots, a_k]$ and $x = [x_1, x_2, \dots, x_k]$. The first tuple is the predefined set; the second tuple, in which x_i is only 0 or 1, defines which elements of a are to be dropped in the knapsack. The sum of elements in the knapsack is

$$s = \text{knapsackSum}(a, x) = x_1a_1 + x_2a_2 + \dots + x_ka_k$$

Given a and x , it is easy to calculate s . However, given s and a it is difficult to find x . In other words, $s = \text{knapsackSum}(x, a)$ is easy to calculate, but $x = \text{inv_knapsackSum}(s, a)$ is difficult. The function *knapsackSum* is a one-way function if a is a general k -tuple.

CRYPTOGRAPHY

Asymmetric-Key Cryptography and Key Management-Unit 3

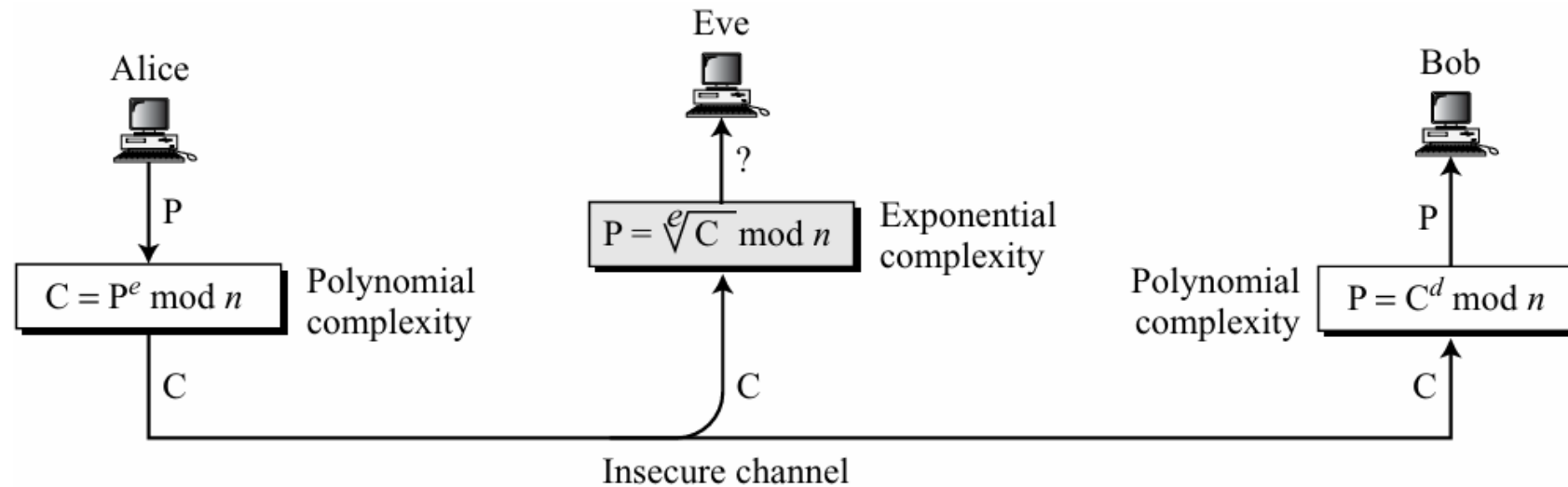
Archana M

Department of Computer Applications

RSA CRYPTOSYSTEM

- The most common public-key algorithm is the RSA cryptosystem, named for its inventors (Rivest, Shamir, and Adleman).
- RSA uses two exponents, e and d , where e is public and d is private. Suppose P is the plaintext and C is the ciphertext. Alice uses $C = P^e \bmod n$ to create ciphertext C from plaintext P .
- Bob use $P = C^d \bmod n$ to retrieve the plaintext sent by Alice
- The modulus n , a very large number, is created during the key generation process.
- Encryption and decryption use modular exponentiation

Figure 10.5 *Complexity of operations in RSA*

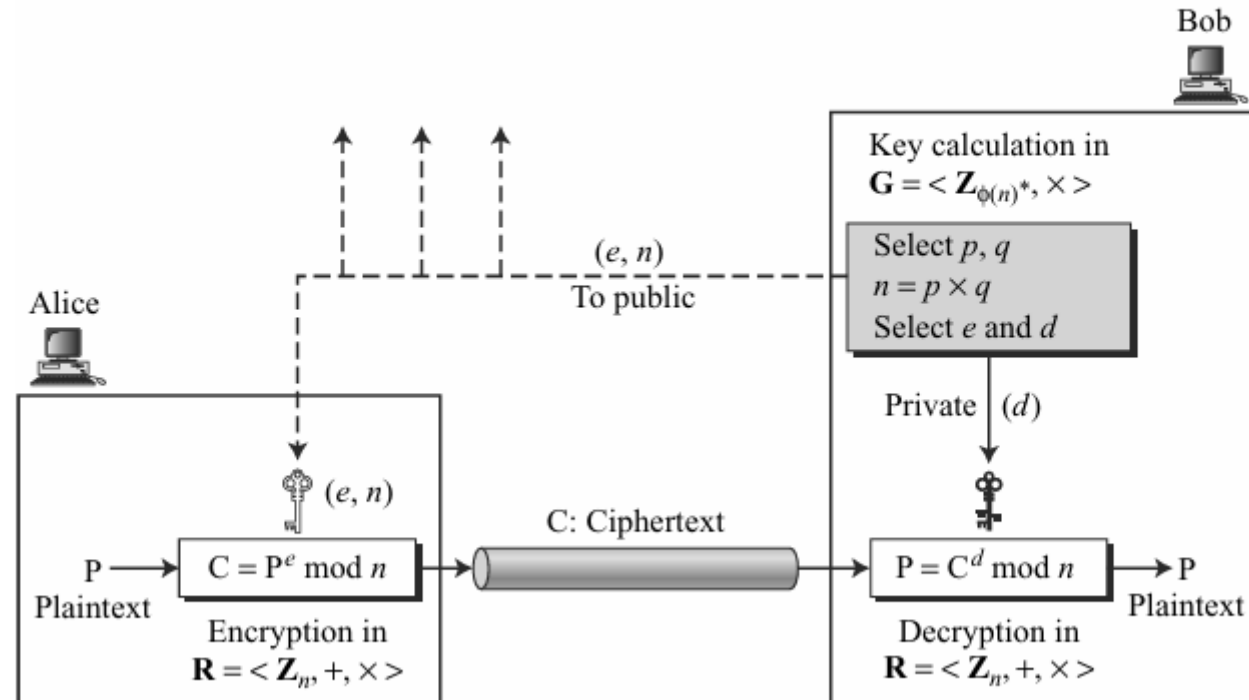


In other words, Alice uses a one-way function (modular exponentiation) with a trapdoor known only to Bob. Eve, who does not know the trapdoor, cannot decrypt the message. If some day, a polynomial algorithm for e th root modulo n calculation is found, modular exponentiation is not a one-way function any more.

Procedure

**RSA uses modular exponentiation for encryption/decryption;
To attack it, Eve needs to calculate $e\sqrt[n]{C} \bmod n$.**

Figure 10.6 Encryption, decryption, and key generation in RSA



Two Algebraic Structures

RSA uses two algebraic structures: a ring and a group.

Encryption/Decryption Ring Encryption and decryption are done using the commutative ring $\mathbf{R} = \langle \mathbf{Z}_n, +, \times \rangle$ with two arithmetic operations: addition and multiplication. In RSA, this ring is public because the modulus n is public. Anyone can send a message to Bob using this ring to do encryption.

Key-Generation Group RSA uses a multiplicative group $\mathbf{G} = \langle \mathbf{Z}_{\phi(n)}^*, \times \rangle$ for key generation. This group supports only multiplication and division (using multiplicative inverses), which are needed for generating public and private keys. This group is hidden from the public because its modulus, $\phi(n)$, is hidden from the public. We will see shortly that if Eve can find this modulus, she can easily attack the cryptosystem.

RSA uses two algebraic structures:
a public ring $R = \langle \mathbb{Z}_n, +, \times \rangle$ and a private group $G = \langle \mathbb{Z}_{\phi(n)}^*, \times \rangle$.

Key Generation

Bob uses the steps shown in Algorithm 10.2 to create his public and private key. After key generation, Bob announces the tuple (e, n) as his public key; Bob keeps the integer d as his private key. Bob can discard p , q , and $\phi(n)$; they will not be needed unless Bob needs to change his private key without changing the modulus (which is not recommended, as we will see shortly). To be secure, the recommended size for each prime, p or q , is 512 bits (almost 154 decimal digits). This makes the size of n , the modulus, 1024 bits (309 digits).

Algorithm 10.2 *RSA Key Generation*

RSA_Key_Generation

```
{  
  Select two large primes  $p$  and  $q$  such that  $p \neq q$ .  
   $n \leftarrow p \times q$   
   $\phi(n) \leftarrow (p - 1) \times (q - 1)$   
  Select  $e$  such that  $1 < e < \phi(n)$  and  $e$  is coprime to  $\phi(n)$   
   $d \leftarrow e^{-1} \bmod \phi(n)$  //  $d$  is inverse of  $e$  modulo  $\phi(n)$   
  Public_key  $\leftarrow (e, n)$  // To be announced publicly  
  Private_key  $\leftarrow d$  // To be kept secret  
  return Public_key and Private_key  
}
```

In RSA, the tuple (e, n) is the public key; the integer d is the private key.

Encryption

- Anyone can send a message to Bob using his public key.
- Encryption in RSA can be done using an algorithm with polynomial time complexity, as shown in Algorithm 10.3.
- The size of the plaintext must be less than n , which means that if the size of the plaintext is larger than n , it should be divided into blocks.

Algorithm 10.3 *RSA encryption*

```
RSA_Encryption ( $P, e, n$ )           //  $P$  is the plaintext in  $Z_n$  and  $P < n$   
{  
     $C \leftarrow$  Fast_Exponentiation ( $P, e, n$ )    // Calculation of  $(P^e \bmod n)$   
    return  $C$   
}
```

- Decryption

Bob can use Algorithm 10.4 to decrypt the ciphertext message he received. Decryption in RSA can be done using an algorithm with polynomial time complexity. The size of the ciphertext is less than n .

Algorithm 10.4 *RSA decryption*

RSA_Decryption (C, d, n)	// C is the ciphertext in Z_n
{	
$P \leftarrow$ Fast_Exponentiation (C, d, n)	// Calculation of $(C^d \bmod n)$
return P	
}	

In RSA, p and q must be at least 512 bits; n must be at least 1024 bits.

Some Trivial Example

Bob chooses 7 and 11 as p and q and calculates $n = 7 \times 11 = 77$. The value of $\phi(n) = (7 - 1)(11 - 1)$ or 60. Now he chooses two exponents, e and d , from $\mathbf{Z}_{60}^*$. If he chooses e to be 13, then d is 37. Note that $e \times d \bmod 60 = 1$ (they are inverses of each other). Now imagine that Alice wants to send the plaintext 5 to Bob. She uses the public exponent 13 to encrypt 5.

Plaintext: 5

$$C = 5^{13} = 26 \bmod 77$$

Ciphertext: 26

Bob receives the ciphertext 26 and uses the private key 37 to decipher the ciphertext:

Ciphertext: 26

$$P = 26^{37} = 5 \bmod 77$$

Plaintext: 5

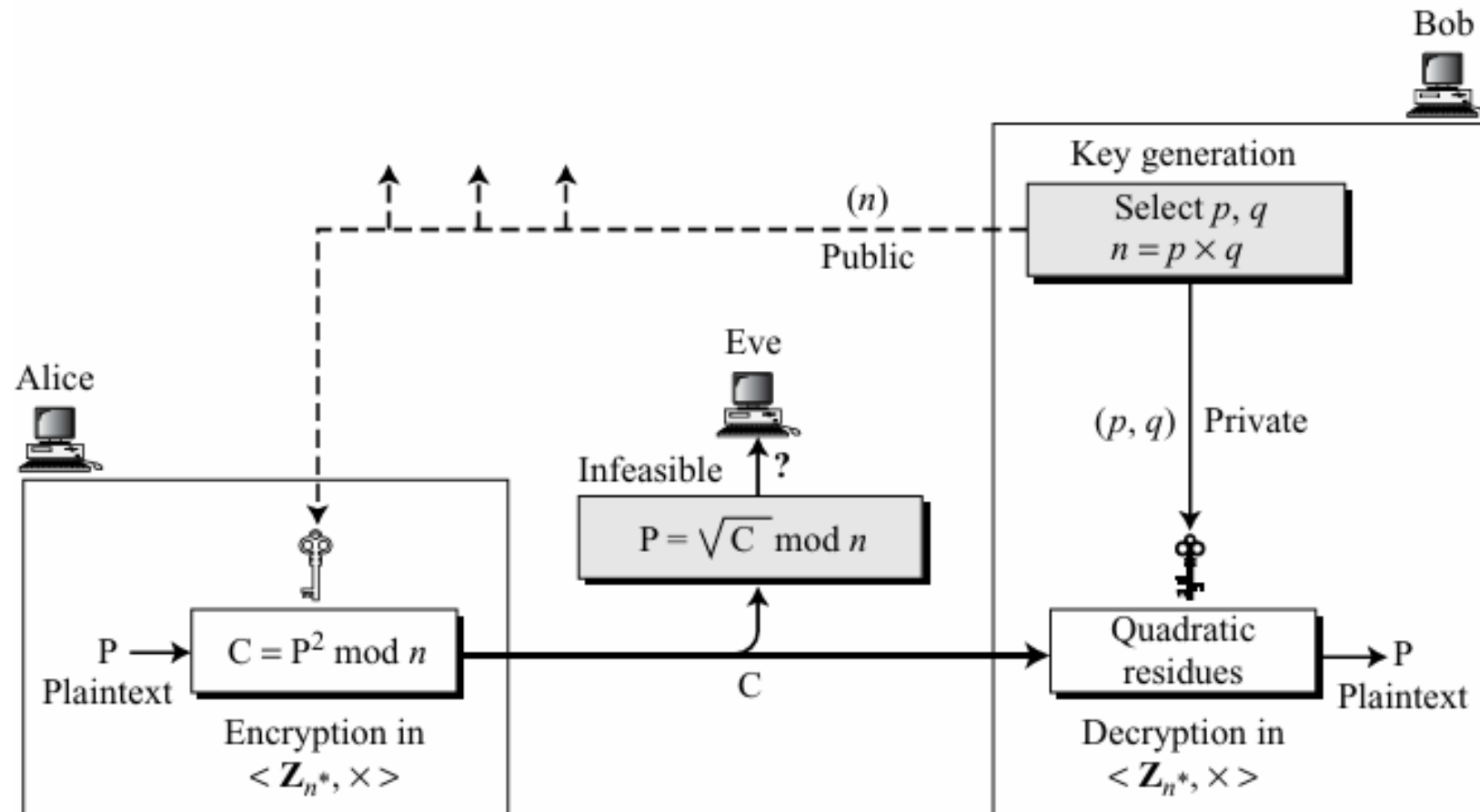
The plaintext 5 sent by Alice is received as plaintext 5 by Bob.

RABIN CRYPTOSYSTEM

- The Rabin cryptosystem, devised by M. Rabin, is a variation of the RSA cryptosystem.
- RSA is based on the exponentiation congruence;
- Rabin is based on quadratic congruence. T
- The Rabin cryptosystem can be thought of as an RSA cryptosystem in which the value of e and d are fixed; $e = 2$ and $d = \frac{1}{2}$
- In other words, the encryption is $C \equiv P^2 \pmod{n}$ and the decryption is $P \equiv C^{1/2} \pmod{n}$.

-
- The public key in the Rabin cryptosystem is n ; the private key is the tuple (p, q) .
 - Everyone can encrypt a message using n ; only Bob can decrypt the message using p and q .
 - Decryption of the message is infeasible for Eve because she does not know the values of p and q

Figure 10.10 *Encryption, decryption, and key generation in the Rabin cryptosystem*



- We need to emphasize a point here.
- If Bob is using RSA, he can keep d and n and discard p , q , and $\phi(n)$ after key generation.
- If Bob is using Rabin cryptosystem, he needs to keep p and q .

Key Generation: Bob uses the steps shown in Algorithm 10.6 to create his public key and private key.

Algorithm 10.6 *Key generation for Rabin cryptosystem*

Rabin_Key_Generation

```
{  
    Choose two large primes  $p$  and  $q$  in the form  $4k + 3$  and  $p \neq q$ .  
     $n \leftarrow p \times q$   
    Public_key  $\leftarrow n$                                 // To be announced publicly  
    Private_key  $\leftarrow (p, q)$                         // To be kept secret  
    return Public_key and Private_key  
}
```

- Encryption

Anyone can send a message to Bob using his public key. The encrypting process is shown

```
Rabin_Encryption ( $n$ ,  $P$ )           //  $n$  is the public key;  $P$  is the ciphertext from  $\mathbf{Z}_n^*$   
{  
     $C \leftarrow P^2 \bmod n$            //  $C$  is the ciphertext  
    return  $C$   
}
```

Although the plaintext P can be chosen from the set $\mathbf{Z}_n$, we have defined the set to be in $\mathbf{Z}_n^*$ to make the decryption easier.

Encryption in the Rabin cryptosystem is very simple. The operation needs only one multiplication, which can be done quickly. This is beneficial when resources are limited. For example, smart cards have limited memory and need to use short CPU time.

Decryption

```
Rabin_Decryption ( $p, q, C$ )           //  $C$  is the ciphertext;  $p$  and  $q$  are private keys
{
     $a_1 \leftarrow +(C^{(p+1)/4}) \bmod p$ 
     $a_2 \leftarrow -(C^{(p+1)/4}) \bmod p$ 
     $b_1 \leftarrow +(C^{(q+1)/4}) \bmod q$ 
     $b_2 \leftarrow -(C^{(q+1)/4}) \bmod q$ 
    // The algorithm for the Chinese remainder theorem is called four times.
     $P_1 \leftarrow \text{Chinese\_Remainder}(a_1, b_1, p, q)$ 
     $P_2 \leftarrow \text{Chinese\_Remainder}(a_1, b_2, p, q)$ 
     $P_3 \leftarrow \text{Chinese\_Remainder}(a_2, b_1, p, q)$ 
     $P_4 \leftarrow \text{Chinese\_Remainder}(a_2, b_2, p, q)$ 
    return  $P_1, P_2, P_3$ , and  $P_4$ 
}
```

CRYPTOGRAPHY

Asymmetric-Key Cryptography and Key Management-Unit 3

Archana M

Department of Computer Applications

Key Management

- Previous chapters have discussed symmetric-key and asymmetric-key cryptography.
- However, we have not yet discussed how secret keys in symmetric-key cryptography, and public keys in asymmetric-key cryptography, are distributed and maintained.

SYMMETRIC-KEY DISTRIBUTION

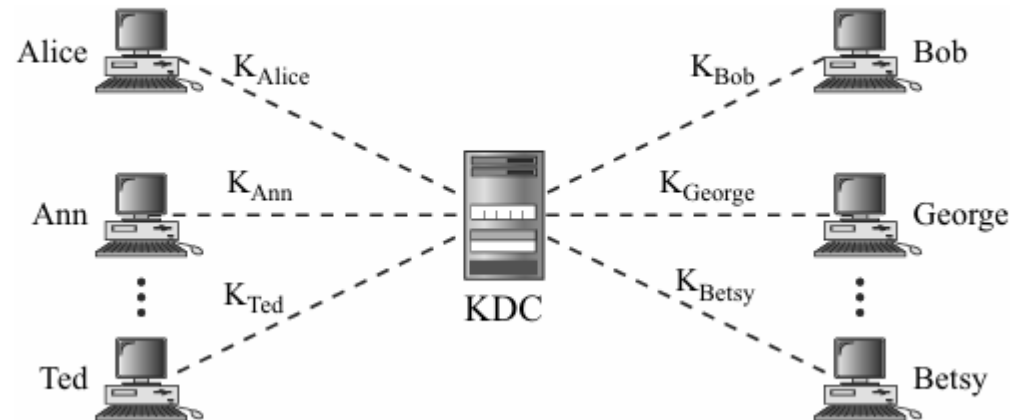
- Symmetric-key cryptography is more efficient than asymmetric-key cryptography for enciphering large messages.
- Symmetric-key cryptography, however, needs a shared secret key between two parties.
- If Alice needs to exchange confidential messages with N people, she needs N different keys.
- What if N people need to communicate with each other?
- A total of $N(N - 1)$ keys is needed if we require that Alice and Bob use two keys for bidirectional communication;
- Only $N(N - 1)/2$ keys are needed if we allow a key to be used for both directions.

-
- This means that if one million people need to communicate with each other, each person has almost one million different keys;
 - In total, almost one trillion keys are needed.
 - This is normally referred to as the N^2 problem because the number of required keys for N entities is N^2 .
 - The number of keys is not the only problem; the distribution of keys is another.
 - If Alice and Bob want to communicate, they need a way to exchange a secret key;
 - if Alice wants to communicate with one million people, how can she exchange one million keys with one million people?
 - Using the Internet is definitely not a secure method. It is obvious that we need an efficient way to maintain and distribute secret keys.

Key-Distribution Center: KDC

- A practical solution is the use of a trusted third party, referred to as a key-distribution center (KDC).
- To reduce the number of keys, each person establishes a shared secret key with the KDC,

Figure 15.1 *Key-distribution center (KDC)*



Session Keys

- A KDC creates a secret key for each member.
- This secret key can be used only between the member and the KDC, not between two members.
- If Alice needs to communicate secretly with Bob, she needs a secret key between herself and Bob.
- A KDC can create a session key between Alice and Bob, using their keys with the center.
- The keys of Alice and Bob are used to authenticate Alice and Bob to the center and to each other before the session key is established.
- After communication is terminated, the session key is no longer useful.

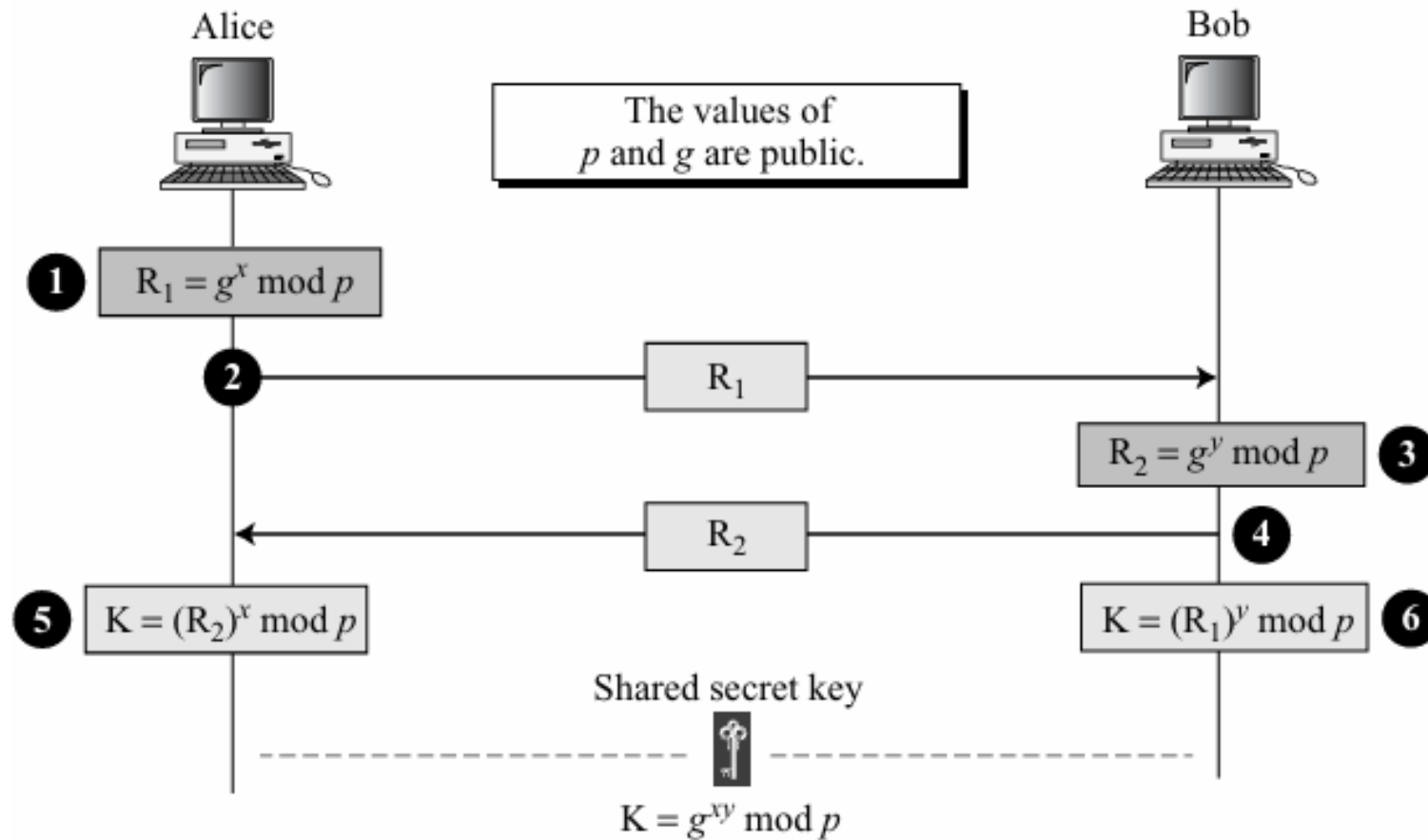
SYMMETRIC-KEY AGREEMENT

- Alice and Bob can create a session key between themselves without using a KDC.
- This method of session-key creation is referred to as the symmetric-key agreement.
- Although there are several ways to accomplish this, only two common methods, :Diffie Hellman and station-to-station will be discussing here

Diffie-Hellman Key Agreement

- In the Diffie-Hellman protocol two parties create a symmetric session key without the need of a KDC.
- Before establishing a symmetric key, the two parties need to choose two numbers p and g .
- The first number, p , is a large prime number on the order of 300 decimal digits (1024 bits)
- The second number, g , is a generator of order $p - 1$ in the group .
These two (group and generator) do not need to be confidential. They can be sent through the Internet; they can be public

Figure 15.9 *Diffie-Hellman method*



The steps are as follows:

1. Alice chooses a large random number x such that $0 \leq x \leq p - 1$ and calculates $R_1 = g^x \bmod p$.
2. Bob chooses another large random number y such that $0 \leq y \leq p - 1$ and calculates $R_2 = g^y \bmod p$.
3. Alice sends R_1 to Bob. Note that Alice does not send the value of x ; she sends only R_1 .
4. Bob sends R_2 to Alice. Again, note that Bob does not send the value of y , he sends only R_2 .
5. Alice calculates $K = (R_2)^x \bmod p$.
6. Bob also calculates $K = (R_1)^y \bmod p$.

K is the symmetric key for the session.

$$K = (g^x \bmod p)^y \bmod p = (g^y \bmod p)^x \bmod p = g^{xy} \bmod p$$

Bob has calculated $K = (R_1)^y \bmod p = (g^x \bmod p)^y \bmod p = g^{xy} \bmod p$. Alice has calculated $K = (R_2)^x \bmod p = (g^y \bmod p)^x \bmod p = g^{xy} \bmod p$. Both have reached the same value without Bob knowing the value of x and without Alice knowing the value of y .

The symmetric (shared) key in the Diffie-Hellman method is $K = g^{xy} \bmod p$.

- Example:

Let us give a trivial example to make the procedure clear. Our example uses small numbers, but note that in a real situation, the numbers are very large. Assume that $g = 7$ and $p = 23$. The steps are as follows:

1. Alice chooses $x = 3$ and calculates $R_1 = 7^3 \bmod 23 = 21$.
 2. Bob chooses $y = 6$ and calculates $R_2 = 7^6 \bmod 23 = 4$.
 3. Alice sends the number 21 to Bob.
 4. Bob sends the number 4 to Alice.
 5. Alice calculates the symmetric key $K = 4^3 \bmod 23 = 18$.
 6. Bob calculates the symmetric key $K = 21^6 \bmod 23 = 18$.
- The value of K is the same for both Alice and Bob; $g^{xy} \bmod p = 7^{18} \bmod 23 = 18$.

- Example 2:

Let us give a more realistic example. We used a program to create a random integer of 512 bits (the ideal is 1024 bits). The integer p is a 159-digit number. We also choose g , x , and y as shown below:

p	764624298563493572182493765955030507476338096726949748923573772860925 235666660755423637423309661180033338106194730130950414738700999178043 6548785807987581
g	2
x	557
y	273

The following shows the values of R_1 , R_2 , and K .

R_1	844920284205665505216172947491035094143433698520012660862863631067673 619959280828586700802131859290945140217500319973312945836083821943065 966020157955354
R_2	435262838709200379470747114895581627636389116262115557975123379218566 310011435718208390040181876486841753831165342691630263421106721508589 6255201288594143
K	155638000664522290596225827523270765273218046944423678520320400146406 500887936651204257426776608327911017153038674561252213151610976584200 1204086433617740

CRYPTOGRAPHY

Asymmetric-Key Cryptography and Key Management-Unit 3

Archana M

Department of Computer Applications

Key Management

- Previous chapters have discussed symmetric-key and asymmetric-key cryptography.
- However, we have not yet discussed how secret keys in symmetric-key cryptography, and public keys in asymmetric-key cryptography, are distributed and maintained.

SYMMETRIC-KEY DISTRIBUTION

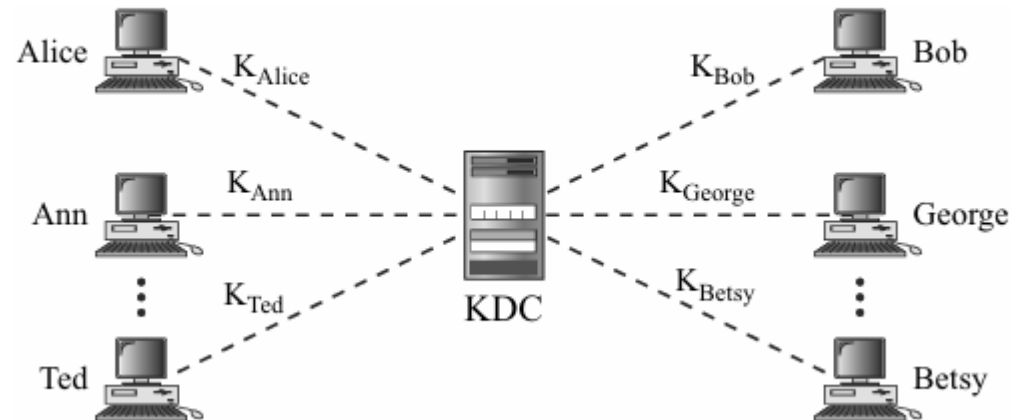
- Symmetric-key cryptography is more efficient than asymmetric-key cryptography for enciphering large messages.
- Symmetric-key cryptography, however, needs a shared secret key between two parties.
- If Alice needs to exchange confidential messages with N people, she needs N different keys.
- What if N people need to communicate with each other?
- A total of $N(N - 1)$ keys is needed if we require that Alice and Bob use two keys for bidirectional communication;
- Only $N(N - 1)/2$ keys are needed if we allow a key to be used for both directions.

- This means that if one million people need to communicate with each other, each person has almost one million different keys;
- In total, almost one trillion keys are needed.
- This is normally referred to as the N^2 problem because the number of required keys for N entities is N^2 .
- The number of keys is not the only problem; the distribution of keys is another.
- If Alice and Bob want to communicate, they need a way to exchange a secret key;
- if Alice wants to communicate with one million people, how can she exchange one million keys with one million people?
- Using the Internet is definitely not a secure method. It is obvious that we need an efficient way to maintain and distribute secret keys.

Key-Distribution Center: KDC

- A practical solution is the use of a trusted third party, referred to as a key-distribution center (KDC).
- To reduce the number of keys, each person establishes a shared secret key with the KDC,

Figure 15.1 *Key-distribution center (KDC)*



Session Keys

- A KDC creates a secret key for each member.
- This secret key can be used only between the member and the KDC, not between two members.
- If Alice needs to communicate secretly with Bob, she needs a secret key between herself and Bob.
- A KDC can create a session key between Alice and Bob, using their keys with the center.
- The keys of Alice and Bob are used to authenticate Alice and Bob to the center and to each other before the session key is established.
- After communication is terminated, the session key is no longer useful.

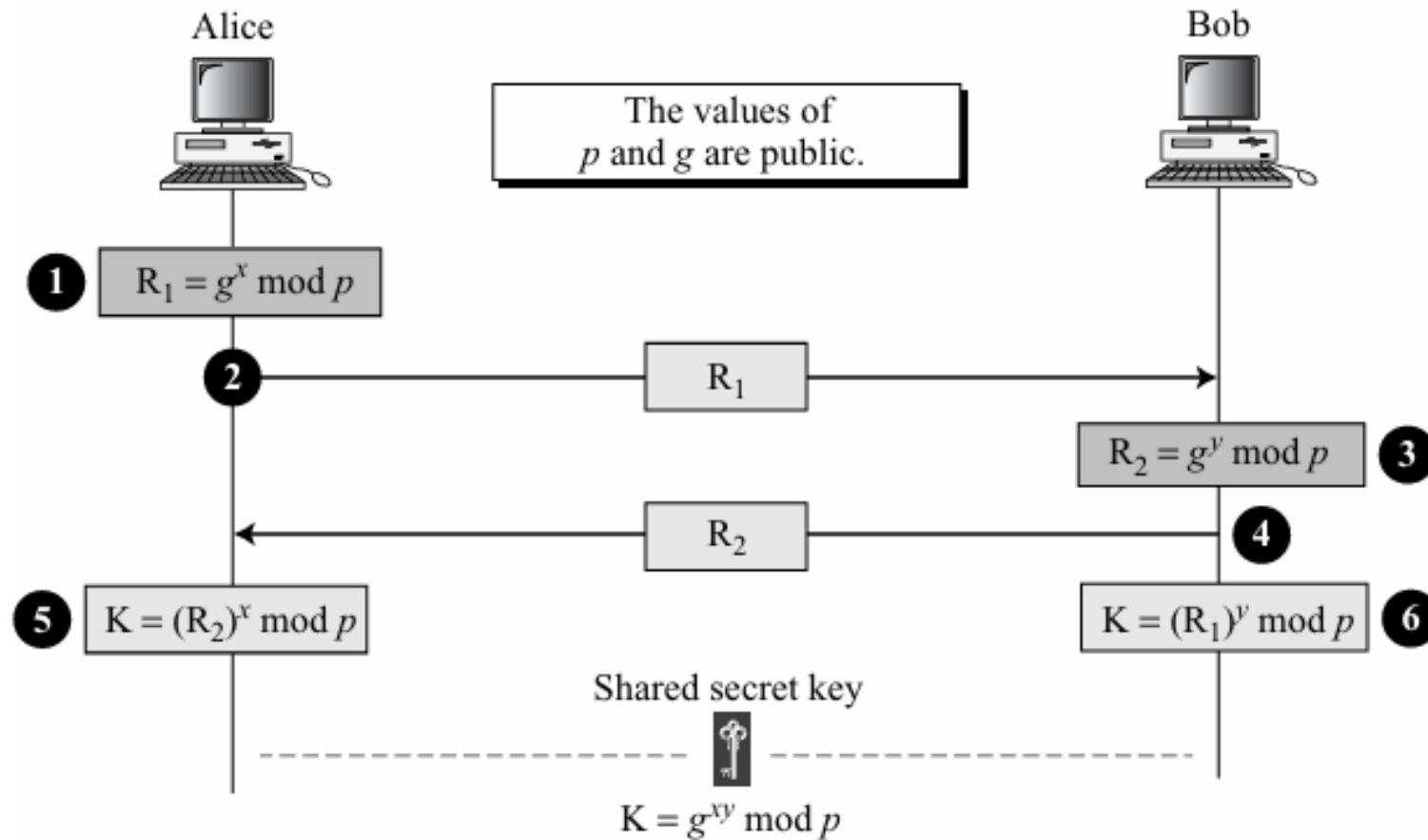
SYMMETRIC-KEY AGREEMENT

- Alice and Bob can create a session key between themselves without using a KDC.
- This method of session-key creation is referred to as the symmetric-key agreement.
- Although there are several ways to accomplish this, only two common methods, :Diffie Hellman and station-to-station will be discussing here

Diffie-Hellman Key Agreement

- In the Diffie-Hellman protocol two parties create a symmetric session key without the need of a KDC.
- Before establishing a symmetric key, the two parties need to choose two numbers p and g .
- The first number, p , is a large prime number on the order of 300 decimal digits (1024 bits)
- The second number, g , is a generator of order $p - 1$ in the group .
These two (group and generator) do not need to be confidential. They can be sent through the Internet; they can be public

Figure 15.9 *Diffie-Hellman method*



The steps are as follows:

1. Alice chooses a large random number x such that $0 \leq x \leq p - 1$ and calculates $R_1 = g^x \bmod p$.
2. Bob chooses another large random number y such that $0 \leq y \leq p - 1$ and calculates $R_2 = g^y \bmod p$.
3. Alice sends R_1 to Bob. Note that Alice does not send the value of x ; she sends only R_1 .
4. Bob sends R_2 to Alice. Again, note that Bob does not send the value of y , he sends only R_2 .
5. Alice calculates $K = (R_2)^x \bmod p$.
6. Bob also calculates $K = (R_1)^y \bmod p$.

K is the symmetric key for the session.

$$K = (g^x \bmod p)^y \bmod p = (g^y \bmod p)^x \bmod p = g^{xy} \bmod p$$

Bob has calculated $K = (R_1)^y \bmod p = (g^x \bmod p)^y \bmod p = g^{xy} \bmod p$. Alice has calculated $K = (R_2)^x \bmod p = (g^y \bmod p)^x \bmod p = g^{xy} \bmod p$. Both have reached the same value without Bob knowing the value of x and without Alice knowing the value of y .

The symmetric (shared) key in the Diffie-Hellman method is $K = g^{xy} \bmod p$.

- Example:

Let us give a trivial example to make the procedure clear. Our example uses small numbers, but note that in a real situation, the numbers are very large. Assume that $g = 7$ and $p = 23$. The steps are as follows:

1. Alice chooses $x = 3$ and calculates $R_1 = 7^3 \bmod 23 = 21$.
 2. Bob chooses $y = 6$ and calculates $R_2 = 7^6 \bmod 23 = 4$.
 3. Alice sends the number 21 to Bob.
 4. Bob sends the number 4 to Alice.
 5. Alice calculates the symmetric key $K = 4^3 \bmod 23 = 18$.
 6. Bob calculates the symmetric key $K = 21^6 \bmod 23 = 18$.
- The value of K is the same for both Alice and Bob; $g^{xy} \bmod p = 7^{18} \bmod 23 = 18$.

- Example 2:

Let us give a more realistic example. We used a program to create a random integer of 512 bits (the ideal is 1024 bits). The integer p is a 159-digit number. We also choose g , x , and y as shown below:

p	764624298563493572182493765955030507476338096726949748923573772860925 235666660755423637423309661180033338106194730130950414738700999178043 6548785807987581
g	2
x	557
y	273

The following shows the values of R_1 , R_2 , and K .

R_1	844920284205665505216172947491035094143433698520012660862863631067673 619959280828586700802131859290945140217500319973312945836083821943065 966020157955354
R_2	435262838709200379470747114895581627636389116262115557975123379218566 310011435718208390040181876486841753831165342691630263421106721508589 6255201288594143
K	155638000664522290596225827523270765273218046944423678520320400146406 500887936651204257426776608327911017153038674561252213151610976584200 1204086433617740

CRYPTOGRAPHY

Asymmetric-Key Cryptography and Key Management-Unit 3

Archana M

Department of Computer Applications

Analysis of Diffie-Hellman

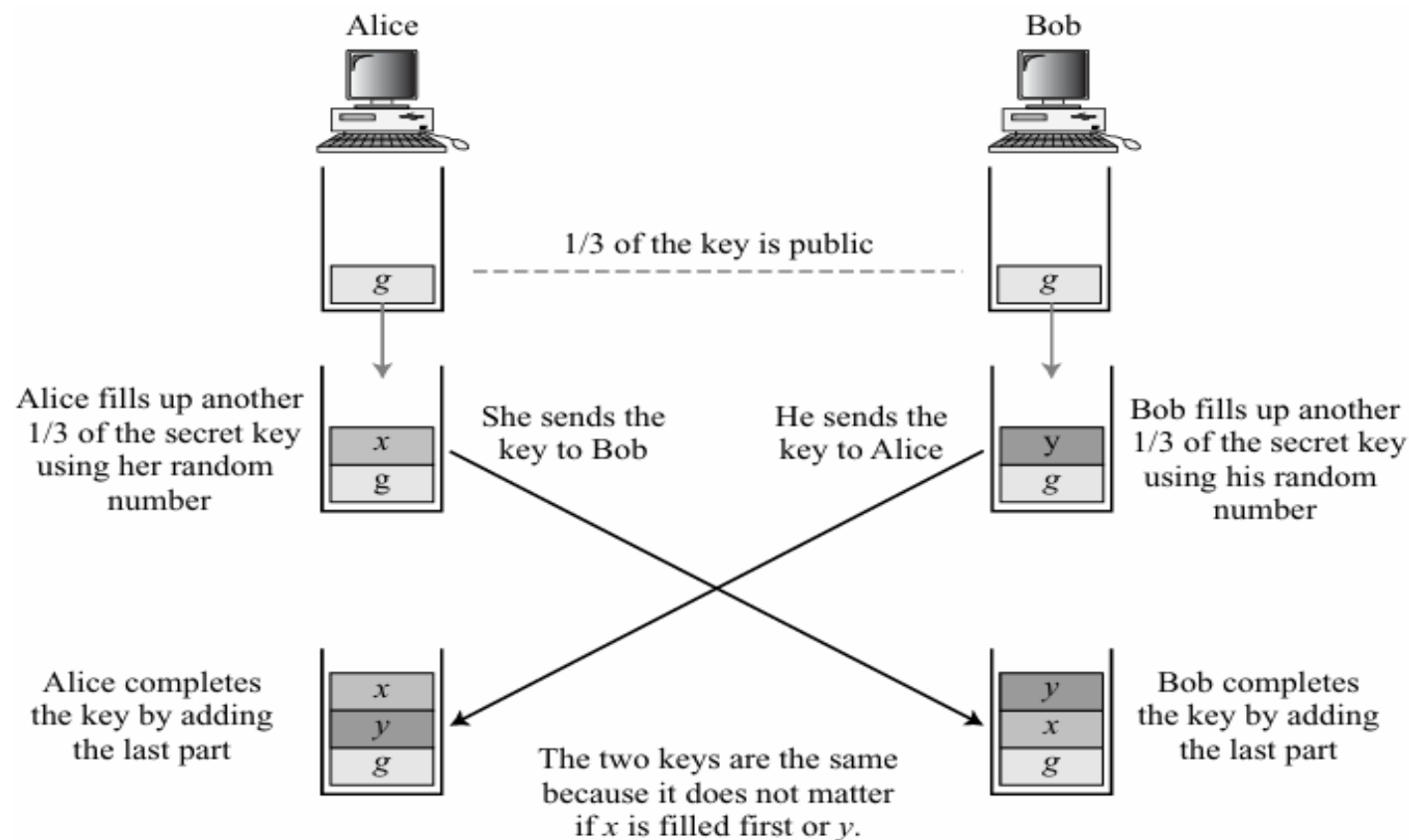
- The Diffie-Hellman concept, shown is simple but elegant.
- We can think of the secret key between Alice and Bob as made of three parts: g , x , and y .
- The first part is public. Everyone knows $1/3$ of the key; g is a public value.
- The other two parts must be added by Alice and Bob.
- Each of them add one part. Alice adds x as the second part for Bob;
- Bob adds y as the second part for Alice

-
- When Alice receives the 2/3 completed key from Bob, she adds the last part, her x , to complete the key.
 - When Bob receives the 2/3-completed key from Alice, he adds the last part, his y , to complete the key.
 - Note that although the key in Alice's hand consists of g , y , and x and the key in Bob's hand consists of g , x , and y , these two keys are the same because $g^{xy} = g^{yx}$.

Security of Diffie-Hellman

- The Diffie-Hellman key exchange is susceptible to two attacks: the discrete logarithm attack and the man-in-the-middle attack

Figure 15.10 *Diffie-Hellman idea*



-
- Discrete Logarithm Attack The security of the key exchange is based on the difficulty of the discrete logarithm problem.
 - Eve can intercept R_1 and R_2 .
 - If she can find x from $R_1 = gx \bmod p$ and y from $R_2 = gy \bmod p$, then she can calculate the symmetric key $K = g^{xy} \bmod p$.
 - The secret key is not secret anymore.
 - To make Diffie-Hellman safe from the discrete logarithm attack, the following are recommended.

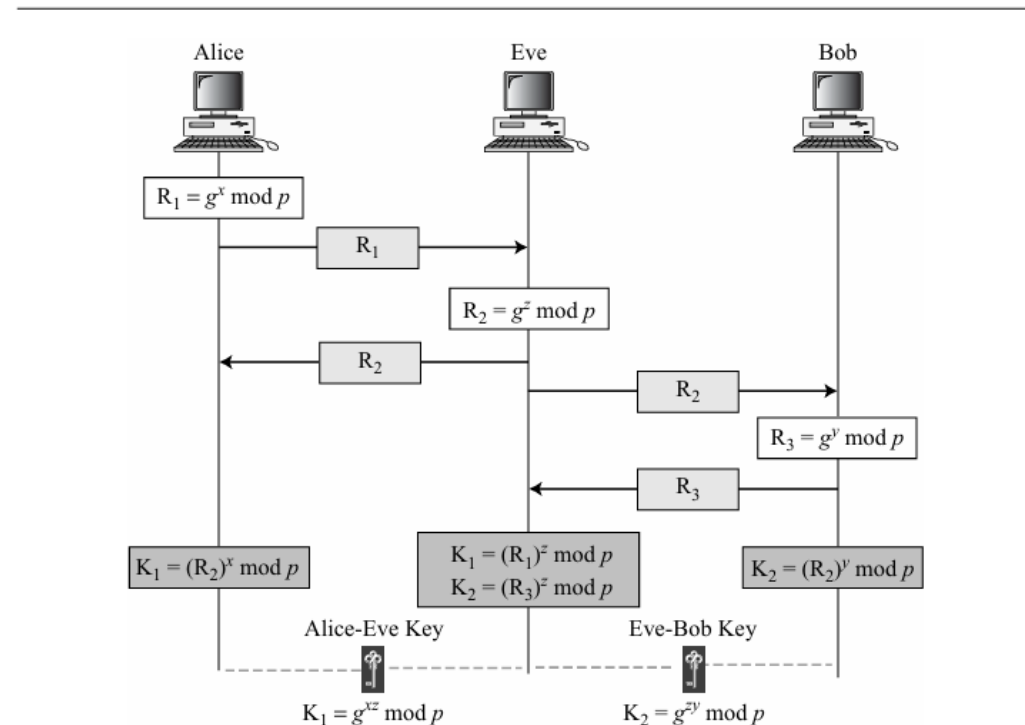
1. The prime p must be very large (more than 300 decimal digits).
2. The prime p must be chosen such that $p - 1$ has at least one large prime factor (more than 60 decimal digits).
3. The generator must be chosen from the group $\langle \mathbf{Z}_{p^*}, \times \rangle$.
4. Bob and Alice must destroy x and y after they have calculated the symmetric key. The values of x and y must be used only once.

Man-in-the-Middle Attack

The protocol has another weakness.

Eve does not have to find the value of x and y to attack the protocol.

She can fool Alice and Bob by creating two keys: one between herself and Alice, and another between herself and Bob. Figure 15.11 shows the situation.



The following can happen:

1. Alice chooses x , calculates $R_1 = g^x \bmod p$, and sends R_1 to Bob.
2. Eve, the intruder, intercepts R_1 . She chooses z , calculates $R_2 = g^z \bmod p$, and sends R_2 to both Alice and Bob.
3. Bob chooses y , calculates $R_3 = g^y \bmod p$, and sends R_3 to Alice. R_3 is intercepted by Eve and never reaches Alice.
4. Alice and Eve calculate $K_1 = g^{xz} \bmod p$, which becomes a shared key between Alice and Eve. Alice, however, thinks that it is a key shared between Bob and herself.
5. Eve and Bob calculate $K_2 = g^{zy} \bmod p$, which becomes a shared key between Eve and Bob. Bob, however, thinks that it is a key shared between Alice and himself.

-
- In other words, two keys, instead of one, are created: one between Alice and Eve, one between Eve and Bob.
 - When Alice sends data to Bob encrypted with K (shared by Alice and Eve), it can be deciphered and read by Eve.
 - Eve can send the message to Bob encrypted by K2 (shared key between Eve and Bob); or she can even change the message or send a totally new message.
 - Bob is fooled into believing that the message has come from Alice.
 - A similar scenario can happen to Alice in the other direction.

-
- This situation is called a man-in-the-middle attack because Eve comes in between and intercepts R 1 , sent by Alice to Bob, and R 3 , sent by Bob to Alice.
 - It is also known as a ***bucket brigade attack*** because it resembles a short line of volunteers passing a bucket of water from person to person.



THANK YOU

Archana M

Department of Computer Applications

archanam@pes.edu